PRACTICE WORKSHOP

Workshop 4: Inheritance

This workshop builds class hierarchies where shared behavior lives in a superclass and specialized behavior lives in subclasses.

Learning Objectives

- Create a subclass that extends a superclass and reuses its fields/methods.
- Use super() to call a superclass constructor.
- Override a method and call the superclass version with super.method().
- Decide when a class hierarchy is the right tool versus composition.

Exercises

Exercise 1: Shape Hierarchy

Create an abstract Shape class with an abstract method getArea(). Create Circle and Rectangle subclasses that implement getArea() correctly. Write a main method that stores several shapes in an array and prints each area.

Hints:

- An abstract class can still have concrete fields and constructors.
- Use Math.PI for the circle.

Exercise 2: Employee / Manager

Create an Employee class with name and baseSalary. Create a Manager subclass that adds a bonus field and overrides a getTotalPay() method to return baseSalary + bonus, while Employee's getTotalPay() simply returns baseSalary.

Hints:

- Manager's constructor should call super(name, baseSalary) before setting its own field.

```
public class Employee {
    protected String name;
    protected double baseSalary;

    public Employee(String name, double baseSalary) {
        this.name = name;
        this.baseSalary = baseSalary;
    }

    public double getTotalPay() {
        return baseSalary;
    }
}

public class Manager extends Employee {
    private double bonus;

    public Manager(String name, double baseSalary, double bonus) {
        // TODO: call super() and set bonus
    }

    // TODO: override getTotalPay()
}
```

Exercise 3: Vehicle Fleet

Model a Vehicle base class with a describe() method, and Car and Motorcycle subclasses that override describe() to add vehicle-specific details while still including the base description via super.describe().

Hints:

- Concatenate the super call's return value with the subclass-specific text.

Stretch Goal (optional)

Add a Van subclass of Car that overrides getCargoCapacity(), and discuss whether Van should instead extend Vehicle directly. Justify your answer in a short comment.